

NETWORK FORENSICS INVESTIGATION REPORT

SBT-DF203: Basic Networking Skills for Digital Forensics
Lab 2 — HTTP Analysis Using Wireshark: Embedded Image Traffic

Report Reference	SBT-DF203-Lab2-2025FWSD11459-AdeleyeAdemiluyi
Course	SBT-DF203: Basic Networking Skills for Digital Forensics
Lab Number	Lab 2 — HTTP Analysis: Embedded Image Traffic
Delivery Block	1/3 Scheduled: 5-11 September 2026
Investigator	Adeleye Ademiluyi Timilehin
Institution	International Cybersecurity and Digital Forensics Academy (ICDFA)
Registration No.	2025/FWSD/11459
Analysis Platform	Kali Linux VM — Apache2, tshark, curl
Date of Analysis	2026-09-11
Submission File	SBT-DF203-Lab2_2025FWSD11459_AdeleyeAdemiluyi.pdf

Academic Integrity Declaration: I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.

1. Executive Summary

This report documents a controlled network forensic investigation of plaintext HTTP traffic containing both an HTML page and an embedded JPEG image. The investigation was conducted as part of the IC DFA SBT-DF203 Lab 2 practical assessment on HTTP Analysis Using Wireshark: Embedded Image Traffic.

A local Apache web server was configured to serve a training webpage (image.html) containing an embedded JPEG image (lab_image.jpg). HTTP traffic was captured using tshark on the loopback interface, producing a 20-packet PCAP file. Analysis confirmed two separate TCP connections and two HTTP GET requests — one for the HTML page and one for the embedded image. The image was successfully exported from the capture and verified against the original using SHA-256 hashing.

2. Evidence Details and Chain of Custody

Field	Value
Case/Lab Identifier	SBT-DF203-Lab2-2025FWSD11459-AdeleyeAdemiluyi
Trainee Name	Adeleye Ademiluyi Timilehin
Date and Time Started	2026-09-10 17:28 EDT
Evidence File	basic.pcapng — 20 packets captured on loopback interface
Source/Generation Method	tshark -i lo -f 'tcp port 80' during controlled curl requests
Original SHA-256 (basic.pcapng)	e9234eba629f49b849b734fd87dbbd6f3ef112f7f48e31dfd796bdc4be429b5b
Working Copy SHA-256	e9234eba629f49b849b734fd87dbbd6f3ef112f7f48e31dfd796bdc4be429b5b
Original Image SHA-256	f2287d34ca12a3a19b38f186e52c7cc1c739aa70d8affa5f84449829acba9a38
Analysis Workstation	Kali Linux VM — tshark, curl, Apache2
Notes	All analysis performed on working copy. Original PCAP preserved unmodified.

3. Environment and Setup

3.1 Folder Structure

Command:

```
mkdir -p ~/SBT-DF203-Lab2/{evidence,working,exported,reports,screenshots,scripts}
```

Folder	Purpose
evidence/	Original PCAP and source image — preserved unmodified
working/	Working copy of PCAP for analysis
exported/	HTTP objects exported from the capture
reports/	tshark command output files and hash records
screenshots/	Evidence screenshots
scripts/	Reserved for analysis scripts

3.2 Web Server Configuration

Apache2 was started and configured to serve two files: image.html (the training webpage) and lab_image.jpg (a 400x200 pixel JPEG created with ImageMagick). The webpage was verified in Firefox at <http://127.0.0.1/image.html> showing the student name and registration number.

3.3 Source File Hashes

File	Size	SHA-256 Hash
lab_image.jpg (original)	13,850 bytes	f2287d34ca12a3a19b38f186e52c7cc1c739aa70d8affa5f84449829acba9a38
image.html	216 bytes	Verified via HTTP Content-Length header
basic.pcapng (original)	20 packets	e9234eba629f49b849b734fd87dbbd6f3ef112f7f48e31dfd796bdc4be429b5b
basic_working.pcapng	20 packets	e9234eba629f49b849b734fd87dbbd6f3ef112f7f48e31dfd796bdc4be429b5b

4. Part A — Web Service Verification

Commands:

```
sudo systemctl status apache2 --no-pager
```

```
sudo ss -lntp | grep ':80'
```

```
curl -v http://127.0.0.1/image.html
```

Check	Result
Apache service status	active (running) — started successfully
TCP port 80 listener	apache2 confirmed listening on *:80
curl GET /image.html	HTTP/1.1 200 OK — Content-Length: 216 — Content-Type: text/html
curl GET /lab_image.jpg	HTTP/1.1 200 OK — Content-Length: 13850 — Content-Type: image/jpeg
Browser rendering	Page displayed correctly with name, reg number and embedded image
Server version	Apache/2.4.68 (Debian)

5. Part B — Traffic Capture

tshark was used to capture loopback traffic on the lo interface filtered to TCP port 80. The capture was started before generating HTTP requests and stopped after both responses were received.

Capture command (Terminal 1):

```
sudo tshark -i lo -f 'tcp port 80' -w /tmp/basic.pcapng
```

Traffic generation (Terminal 2):

```
curl --no-keepalive -v http://127.0.0.1/image.html 2>&1
```

```
curl --no-keepalive -v http://127.0.0.1/lab_image.jpg 2>&1
```

Result: 20 packets captured. The PCAP was moved to the evidence folder and a working copy created. Both copies share identical SHA-256 hashes confirming integrity.

6. Part C — TCP Three-Way Handshake Analysis

Two TCP handshakes were captured — one for each HTTP request. The capture used two separate TCP connections with `--no-keepalive`, demonstrating the browser behaviour of opening a new connection for each object.

Command:

```
tshark -r basic_working.pcapng -Y 'tcp.flags.syn==1' -T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags -e tcp.seq -e tcp.ack
```

Frame	Flag	Source	Src Port	Destination	Dst Port	Seq	Interpretation
1	SYN (0x0002)	127.0.0.1	58036	127.0.0.1	80	0	Client requests TCP session for ima
2	SYN-ACK (0x0012)	127.0.0.1	80	127.0.0.1	58036	0	Server acknowledges, provides ISN
3	ACK (0x0010)	127.0.0.1	58036	127.0.0.1	80	1	Connection established
11	SYN (0x0002)	127.0.0.1	58044	127.0.0.1	80	0	Client requests TCP session for lab_
12	SYN-ACK (0x0012)	127.0.0.1	80	127.0.0.1	58044	0	Server acknowledges second conne
13	ACK (0x0010)	127.0.0.1	58044	127.0.0.1	80	1	Second connection established

Key observation: Two separate TCP connections were opened — client ports 58036 and 58044. This demonstrates that `--no-keepalive` forces a new connection per request, simulating browser behaviour for multiple HTTP objects.

7. Part D — HTTP Request and Response Analysis

7.1 HTTP GET Requests

Frame	Timestamp	Method	URI	Host	User-Agent
4	17:44:01.656	GET	/image.html	127.0.0.1	curl/8.21.0
14	17:44:01.674	GET	/lab_image.jpg	127.0.0.1	curl/8.21.0

7.2 HTTP Responses

Frame	Timestamp	Code	Phrase	Server	Content-Type	Content-Length
6	17:44:01.657	200	OK	Apache/2.4.68	text/html	216 bytes
16	17:44:01.675	200	OK	Apache/2.4.68	image/jpeg	13,850 bytes

7.3 TCP Stream Reconstruction

Stream 0 (image.html): The complete HTTP GET request and 200 OK response were reconstructed showing all headers and the full HTML content including the img tag referencing lab_image.jpg.

Stream 1 (lab_image.jpg): The GET request for the image and the 200 OK response containing the raw JPEG binary data (13,850 bytes) were reconstructed confirming the complete image transfer.

curl vs browser behaviour: curl with --no-keepalive opens a separate TCP connection for each request, similar to a browser loading multiple objects. A real browser would also request the embedded image after parsing the HTML, but may use keep-alive or HTTP/2 multiplexing to reuse the same connection.

8. Part E — Encapsulation, Image Export and Connection Closure

8.1 HTTP Object Export

Command:

```
tshark -r basic_working.pcapng --export-objects "http,/home/kali/ST-DF203-Lab2/exported"
```

Two objects exported:

Exported File	Size	Source
image.html	216 bytes	HTTP response frame 6
lab_image.jpg	13,850 bytes	HTTP response frame 16

8.2 Hash Verification — Recovered Image

File	SHA-256 Hash	Match
lab_image.jpg (original)	f2287d34ca12a3a19b38f186e52c7cc1c739aa70d8affa5f84449829acba9a39	✓
lab_image.jpg (exported)	f2287d34ca12a3a19b38f186e52c7cc1c739aa70d8affa5f84449829acba9a39	✓

Hashes match — the image recovered from the network capture is byte-for-byte identical to the original file served by Apache. This confirms the image was transferred without corruption and can be used as forensic evidence.

8.3 TCP Connection Closure

Frame	Timestamp	Source	Src Port	Flag	Interpretation
8	17:44:01.657	127.0.0.1	58036	FIN-ACK (0x0011)	Client closes connection 1
9	17:44:01.657	127.0.0.1	80	FIN-ACK (0x0011)	Server closes connection 1
18	17:44:01.675	127.0.0.1	58044	FIN-ACK (0x0011)	Client closes connection 2
19	17:44:01.676	127.0.0.1	80	FIN-ACK (0x0011)	Server closes connection 2

Both connections closed gracefully using FIN-ACK exchanges. No RST (reset) packets were observed, confirming normal session termination.

8.4 MAC Address Observation

All frames in the capture show source and destination MAC addresses as 00:00:00:00:00:00. This is because the capture was performed on the loopback interface (lo), which is a virtual interface that does not use real Ethernet hardware. Traffic on the loopback interface never leaves the machine and therefore does not carry real MAC addresses.

For real MAC address evidence, the capture would need to be performed on a physical or virtual Ethernet interface (e.g. eth0) between two separate hosts on the same network segment. A two-VM setup on a host-only network would produce frames with real MAC addresses visible in the Ethernet layer.

9. Required Forensic Findings

Finding	Value/Observation
Capture start time	2026-09-10 17:44:01.655 EDT
Capture end time	2026-09-10 17:44:01.676 EDT
Total packets captured	20 packets
Client IP and port 1	127.0.0.1:58036 (image.html request)
Client IP and port 2	127.0.0.1:58044 (lab_image.jpg request)
Server IP and port	127.0.0.1:80 (Apache)
Client ISN (connection 1)	0 (relative)
Server ISN (connection 1)	0 (relative)
HTTP request 1 timestamp	2026-09-10 17:44:01.656 — GET /image.html
HTTP request 2 timestamp	2026-09-10 17:44:01.674 — GET /lab_image.jpg
Requested URIs	/image.html and /lab_image.jpg
HTTP response codes	200 OK for both requests
Content type — HTML	text/html — 216 bytes
Content type — Image	image/jpeg — 13,850 bytes
Connection termination	Graceful FIN-ACK exchange on both connections
MAC address observation	00:00:00:00:00:00 — loopback interface has no real MAC addresses
Image hash verification	MATCH — exported image identical to original

10. Conclusion

This laboratory exercise successfully demonstrated the analysis of plaintext HTTP traffic containing both an HTML page and an embedded JPEG image. The key findings were:

- Two separate TCP connections were established — one per HTTP object — confirmed by distinct client ports (58036 and 58044)
- Two HTTP GET requests were captured: GET /image.html and GET /lab_image.jpg, both returning HTTP 200 OK
- The image was successfully exported from the PCAP using `tshark --export-objects`
- SHA-256 hash verification confirmed the exported image is byte-for-byte identical to the original (f2287d34...)
- Both TCP connections closed gracefully with FIN-ACK exchanges — no RST packets observed
- Loopback interface captures show MAC addresses as 00:00:00:00:00:00 — real MAC evidence requires a two-VM Ethernet setup
- `curl` with `--no-keepalive` simulates browser behaviour of requesting multiple objects separately
- All original evidence files were preserved unmodified with matching SHA-256 hashes

All commands, outputs, screenshots and recovered files are documented in the accompanying GitHub repository: <https://github.com/Adeleye001/SBT-DF203-Lab2>